
Aliro_Actuator

Mike Jongen

Dec 13, 2023

MODULES

1	Approach 1 (handle)	3
2	Approach 2 (command/response)	5
3	Modules	7
3.1	aliro_actuator	7
3.1.1	aliro_actuator package	7
3.1.1.1	Subpackages	7
3.1.1.2	Submodules	38
3.1.1.3	Module contents	39
4	Indices and tables	41
	Python Module Index	43
	Index	45

The code is separated into 5 parts: the 4 parts as described in the Architecture overview of the specification (chapter 5.1) and the hardware drivers.

This actuator supports both the reader side and the user side of the protocol. The class that implements the reader side is the *Reader*, and the user side is implemented by the *UserDevice*.

Both *Reader* and *UserDevice* have a similar structure. They both have a parameter `transport_protocol` (of type *TransportProtocol*) which indicates which transport protocol to use (NFC or BLE/UWB). After instantiating the class, the `transaction_initiation()` or `transaction_initiation()` method is used to set up the connection to a device. This method blocks until the connection is established.

The *UserDevice* and *Reader* can now be used in one of two ways: using the handle functions for an easy to use approach, or the command/response methods for more control.

APPROACH 1 (HANDLE)

This approach uses the attributes set during instantiation, and the ones received from commands, to make the actuator easier to use. Start by using the `start_new_session()` or `start_new_session()` to start a session. All data received will be stored in this session. The *UserDevice* now needs to wait for a command. This is done with the `wait_for_command()` method, which returns a *Command* to pass to a handle method. Now, use the methods that start with 'handle' to handle the commands:

User methods:

```
handle_select()
handle_auth0()
handle_auth1()
handle_control_flow()
handle_exchange()
handle_load_cert()
```

Reader methods:

```
handle_select()
handle_auth0()
handle_auth1()
handle_control_flow()
handle_exchange()
handle_load_cert()
```


APPROACH 2 (COMMAND/RESPONSE)

This approach gives more control to the programmer. The methods that start with ‘command’/‘response’ require a value for every piece of data they will send. The data received will also need to be manually processed by the programmer (unlike Approach 1, there is no session to save received data). Note that Approach 1 also uses these ‘command’/‘response’ methods, so you can use the Approach 1 methods as an example.

MODULES

3.1 aliro_actuator

3.1.1 aliro_actuator package

3.1.1.1 Subpackages

aliro_actuator.access_protocol package

Submodules

aliro_actuator.access_protocol.apdu module

class `aliro_actuator.access_protocol.apdu.APDU`

Bases: `object`

Class for creating and parsing APDU messages.

Use the `parse_command/response` functions to parse the received commands/responses. Use the **create..._command/response** to create new command/responses. The `as_bytes` attribute of these commands/responses gives a `bytes` which can be send.

has the following attributes: `support_extended_length_apdu`: `bool` (set to `true` to support extended length APDU's, see 8.3 Aliro spec)

create_auth0_command(*transaction_type*: `Transaction`, *transaction_code*: `TransactionCode`,
protocol_version: `int`, *reader_epubk*: `bytes`, *transaction_identifier*: `bytes`,
reader_identifier: `bytes`) → `Command`

create_auth0_response(*endpoint_epubk*: `bytes`, *status*: `int`, *cryptogram*: `bytes` | `None` = `None`) →
`Response`

create_auth1_command(*response*: `Auth1Response`, *request_access_credentials*: `bool`, *reader_sig*: `bytes`,
certificate_data: `bytes` | `None` = `None`) → `Command`

create_auth1_response(key_slot: bytes | None, public_key: bytes | None, expected_response: Auth1Response, signature: bytes, encryption: EncryptionEngine, status: int = StatusBytes.SUCCESS, private_mailbox_data: bytes | None = None, access_doc_retrieve: bool = False, revocation_doc_retrieve: bool = False, step_up_aid_required: bool = False, data_in_mailbox: bool = False, read_mailbox: bool = False, write_mailbox: bool = False, send_issuer_backend: bool = False, send_bound_app: bool = False, update_doc: bool = False, credential_signed_timestamp: bytes | None = None, revocation_signed_timestamp: bytes | None = None) → Response

create_command(cla: int, ins: int, p1: int, p2: int, data: bytes, le: int | None) → Command

Create a command. (the other more specific functions are recommended)

create_control_flow_command(S1: int, S2: int, domain_specific_data: bytes | None = None) → Command

create_control_flow_response(status: int) → Response

create_envelope_command(payload: bytes) → Command

create_envelope_response(payload: bytes | None = None, status: int = StatusBytes.SUCCESS) → Response

create_error_response(status_bytes: int) → Response

create_exchange_command(atomic_session: bool, payload_tlv: TLV, encryption: EncryptionEngine) → Command

create_exchange_response(payload: bytes, encryption: EncryptionEngine, status: int) → Response

create_get_response_command(payload: bytes) → Command

create_get_response_response(payload: bytes, status: int) → Response

create_load_cert_command(compressed_reader_cert: bytes) → Command

create_load_cert_response(status: int) → Response

create_select_command(aid: bytes) → Command

create_select_response(AID: bytes, type: int, expedited_phase_supported_protocol_versions: list[int], maximum_command_apdu: int | None = None, maximum_response_apdu: int | None = None, vendor_specific_tlv: TLV | None = None, status: int = StatusBytes.SUCCESS) → Response

parse_command(command_as_bytes: bytes, encryption: EncryptionEngine | None = None) → Command

Parse a command bytestring. Used to extract info from a received command.

parse_response(response_as_bytes: bytes, ins: INS, encryption: EncryptionEngine | None = None) → Response

Parse a response bytestring. Used to extract info from a received response.

class aliro_actuator.access_protocol.apdu.Auth1Response(value)

Bases: IntEnum

Indicating the type of response requested in a auth1 command. Send with tag 0x41, bit 0. See table 8-10 of the Aliro spec.

ENDPOINT_PUBLIC_KEY = 1

KEY_SLOT = 0

class aliro_actuator.access_protocol.apdu.**Command**

Bases: *Message*

contains an APDU command.

has the following attributes: as_bytes: bytes (full command as bytes) cla: int ins: int p1: int p2: int lc: int data: bytes le: int

classmethod **create_from_bytestring**(bytestring: bytes) → *Command*

Create a Command from a bytestring.

classmethod **create_from_parameters**(cla: int, ins: int, p1: int, p2: int, data: bytes, le: int | None) → *Command*

Create a Command from its fields.

lc is calculated from the data. Set le to None if no le field is required.

parse_as_auth0() → None

Parse this command as a AUTH0 command.

Checks the fields and raises errors for invalid fields. creates the following attributes: command_parameters: int transaction_code: int expedited_phase_protocol_version: int reader_epubk: bytes transaction_identifier: bytes reader_identifier: bytes vendor_specific_extension: bytes | None

parse_as_auth1() → None

Parse this command as a Auth1 command.

Checks the fields and raises errors for invalid fields. creates the following attributes: command_parameters: int expected_response: Auth1Response request_access_credentials: bool reader_signature: bytes certificate_data: bytes | None

parse_as_control_flow() → None

Parse this command as a Control Flow command.

Checks the fields and raises errors for invalid fields.

parse_as_envelope() → None

Parse this command as a Envelope command.

Checks the fields and raises errors for invalid fields.

parse_as_exchange(encryption: *EncryptionEngine* | None = None) → None

Parse this command as a Exchange command.

Checks the fields and raises errors for invalid fields. creates the following attributes: encrypted_payload authentication_tag decrypted_payload atomic_session payload_tlv read_requests write_requests set_requests notify ursk update_doc

parse_as_get_response() → None

Parse this command as a Get Response command.

Checks the fields and raises errors for invalid fields.

parse_as_load_cert() → None

Parse this command as a Load Cert command.

Checks the fields and raises errors for invalid fields.

parse_as_select() → None

Parse this command as a Select command.

Checks the fields and raises errors for invalid fields.

class aliro_actuator.access_protocol.apdu.**INS**(*value*)

Bases: IntEnum

Possible values of the INS field in an APDU message. See Table 8-2 of the Aliro spec.

AUTH0 = 128

AUTH1 = 129

CONTROL_FLOW = 60

ENVELOPE = 195

EXCHANGE = 201

GET_RESPONSE = 192

LOAD_CERT = 209

SELECT = 164

class aliro_actuator.access_protocol.apdu.**Message**

Bases: object

Parent class of Command and Response. Contains functions that are common for both Command and Response.

to_bytes() → bytes

class aliro_actuator.access_protocol.apdu.**Response**

Bases: *Message*

contains an APDU Response.

has the following attributes: *as_bytes*: bytes (full command as bytes) *sw1*: int *sw2*: int *status*: int (combination of *sw1* and *sw2*) *data*: bytes

classmethod **create_from_bytestring**(*bytestring*: bytes) → *Response*

Create a Response from a bytestring.

classmethod **create_from_parameters**(*data*: bytes | None = None, *status*: int = StatusBytes.SUCCESS) → *Response*

Create a Command from its fields.

parse_as_auth0() → None

Parse this response as a Auth0 response.

creates the following attributes: *endpoint_epubk*: bytes *cryptogram*: bytes (if present)

parse_as_auth1(*encryption*: *EncryptionEngine* | None = None) → None

Parse this response as a Auth1 response.

creates the following attributes: *encrypted_payload*: bytes *authentication_tag*: bytes *decrypted_payload*: bytes *payload_tlv*: TLV *key_slot*: bytes | None *endpoint_public_key*: bytes | None *endpoint_signature*: bytes *private_mailbox_data*: bytes | None *signaling_bitmap*: bytes *credential_signed_timestamp*: bytes | None *revocation_signed_timestamp*: bytes | None *access_credential_response*: bytes | None

parse_as_control_flow() → None

Parse this response as a control_flow response.

parse_as_envelope() → None

Parse this response as a envelope response.

parse_as_exchange(*encryption*: [EncryptionEngine](#) | None = None) → None

Parse this response as a Exchange response.

creates the following attributes: encrypted_payload: bytes authentication_tag: bytes decrypted_payload: bytes status_code: bytes read_data: bytes

parse_as_get_response() → None

Parse this response as a get_response response.

parse_as_load_cert() → None

Parse this response as a Auth1 response.

parse_as_select() → None

Parse this response as a Select response.

creates the following attributes: compl_aid: bytes proprietary_tlv: TLV type: bytes expedited_phase_supported_protocol_versions: bytes maximum_command_apdu: int maximum_response_apdu: int vendor_specific_extensions: TLV

property sw1: int

property sw2: int

class aliro_actuator.access_protocol.apdu.S1(*value*)

Bases: IntEnum

Indicating the S1 parameter in a control flow command. Send with tag 0x41. See table 8-13 and 8.5.7.2 of the Aliro spec.

FINISHED_WITH_FAILURE = 0

FINISHED_WITH_SUCCESS = 1

class aliro_actuator.access_protocol.apdu.S2(*value*)

Bases: IntEnum

Indicating the S2 parameter in a control flow command. Send with tag 0x42. See table 8-13 and 8.5.7.3 of the Aliro spec.

NONE = 0

PROTOCOL_VERSION_NOT_SUPPORTED = 39

class aliro_actuator.access_protocol.apdu.StatusBytes(*value*)

Bases: IntEnum

Indicating (some) known values of the status bytes returned in a response.

COMMAND_NOT_ALLOWED = 26880

COMMAND_NOT_COMPLIANT = 26369

CONDITIONS_OF_USE_NOT_SATISFIED = 27013

```
FILE_OR_APP_NOT_FOUND = 27266
FUNCTIONS_IN_CLA_NOT_SUPPORTED = 26624
FUNCTION_NOT_SUPPORTED = 27265
GENERIC_ERROR = 25600
INCORRECT_P1_P2 = 27392
INCORRECT_PARAMETERS_IN_DATA = 27264
INCORRECT_SECURE_MESSAGING_DOS = 27016
INVALID_CLASS = 28160
INVALID_INSTRUCTION = 27904
LOGICAL_CHANNEL_NOT_SUPPORTED = 26753
MEMORY_FAILURE = 25985
NO_PRECISE_DIAGNOSIS = 28160
SECURITY_STATUS_NOT_SATISFIED = 27010
SUCCESS = 36864
WRONG_LENGTH_IN_LC = 26368
```

```
class aliro_actuator.access_protocol.apdu.Transaction(value)
```

Bases: IntEnum

Indicating the transaction type in a auth0 command. See table 8-5 of the Aliro spec.

```
FAST = 1
```

```
STANDARD = 0
```

```
class aliro_actuator.access_protocol.apdu.TransactionCode(value)
```

Bases: IntEnum

Indicating the transaction code in a auth0 command. See table 8-1 and 8-5 of the Aliro spec.

```
DISARM = 3
```

```
DISARM_WITH_FORCE_USER_AUTHENTICATION = 8
```

```
LOCK = 2
```

```
LOCK_USING_RKE = 5
```

```
LOCK_WITH_FORCE_USER_AUTHENTICATION = 7
```

```
UNLOCK = 1
```

```
UNLOCK_USING_RKE = 4
```

```
UNLOCK_WITH_FORCE_USER_AUTHENTICATION = 6
```


aliro_actuator.access_protocol.authentication module

`aliro_actuator.access_protocol.authentication.create_endpoint_authentication`(*reader_identifier*: bytes, *endpoint_epubk*: PublicKey, *reader_epubk*: PublicKey, *transaction_identifier*: bytes) → TLV

`aliro_actuator.access_protocol.authentication.create_reader_authentication`(*reader_identifier*: bytes, *endpoint_epubk*: PublicKey, *reader_epubk*: PublicKey, *transaction_identifier*: bytes) → TLV

aliro_actuator.access_protocol.defines module

`class aliro_actuator.access_protocol.defines.Auth0`

Bases: object

`COMMAND_TAG = 65`

`CRYPTOGRAM_TAG = 157`

`ENDPOINT_EPUBK_TAG = 134`

`ETPV_TAG = 92`

`READER_EPUBK_TAG = 135`

`READER_IDENTIFIER_TAG = 77`

`RE_VENDOR_SPECIFIC_TAG = 178`

`TRANSACTION_CODE_TAG = 66`

`TRANSACTION_ID_TAG = 76`

`VENDOR_SPECIFIC_TAG = 177`

`class aliro_actuator.access_protocol.defines.Auth1`

Bases: object

`ACCESS_RESPONSE_TAG = 147`

`CERTIFICATE_TAG = 144`

`COMMAND_TAG = 65`

`CREDENTIAL_TIMESTAMP_LEN = 20`

CREDENTIAL_TIMESTAMP_TAG = 145

ENDPOINT_EPUBK_LEN = 65

ENDPOINT_EPUBK_TAG = 90

ENDPOINT_SIG_LEN = 64

ENDPOINT_SIG_TAG = 158

KEY_SLOT_LEN = 8

KEY_SLOT_TAG = 78

MAILBOX_DATA_TAG = 75

READER_SIG_TAG = 158

REVOCATION_TIMESTAMP_LEN = 20

REVOCATION_TIMESTAMP_TAG = 146

SIGNALING_BITMAP_TAG = 94

class aliro_actuator.access_protocol.defines.**ControlFlow**

Bases: object

DOMAIN_SPECIFIC_TAG = 67

S1_TAG = 65

S2_TAG = 66

class aliro_actuator.access_protocol.defines.**EndpointAuth**

Bases: object

ENDPOINT_EPUBK_TAG = 134

READER_EPUBK_TAG = 135

READER_IDENTIFIER_TAG = 77

TRANSACTION_IDENTIFIER_TAG = 76

USAGE = b'N\x88{L'

USAGE_TAG = 147

class aliro_actuator.access_protocol.defines.**Exchange**

Bases: object

NOTIFY_TAG = 174

READ_TAG = 135

SET_TAG = 149

UPDATE_DOC_TAG = 129

URSK_TAG = 152

```
WRITE_TAG = 138

class aliro_actuator.access_protocol.defines.ReaderAuth
    Bases: object
    ENDPOINT_EPUBK_TAG = 134
    READER_EPUBK_TAG = 135
    READER_IDENTIFIER_TAG = 77
    TRANSACTION_IDENTIFIER_TAG = 76
    USAGE = b'A]\x95i'
    USAGE_TAG = 147

class aliro_actuator.access_protocol.defines.Select
    Bases: object
    AID_TAG = 132
    ETSPV_TAG = 92
    EXTENDED_INFO_TAG = 32614
    FCI_TAG = 111
    MAX_COMMAND_TAG = 2
    MAX_RESPONSE_TAG = 2
    PROPRIETARY_TAG = 165
    TYPE_TAG = 128
    VENDOR_SPECIFIC_TAG = 179

class aliro_actuator.access_protocol.defines.TransportProtocol(value)
    Bases: IntEnum
    An enumeration.
    BLE_UWB = 1
    NFC = 0
    SOCKET_BLE = 3
    SOCKET_NFC = 2
```

aliro_actuator.access_protocol.encryption module**class** aliro_actuator.access_protocol.encryption.**DeviceType**(*value*)

Bases: Enum

Enumerator, used by the EncryptionEngine, to indicate if the device is a reader or user.

READER = 1**USER** = 2**class** aliro_actuator.access_protocol.encryption.**EncryptionEngine**(*devicetype: DeviceType,*
exchange_SK_reader: bytes,
exchange_SK_device: bytes)

Bases: object

class to encrypt and decrypt messages. Uses AES-256.

Args:

devicetype (DeviceType): indicates if the device creating this class is a user or reader

exchange_SK_reader (bytes): exchange SK Reader as described in Figure 8-13 of the Aliro spec

exchange_SK_device (bytes): exchange SK Device as described in Figure 8-13 of the Aliro spec

check_counters_valid() → bool

Checks the counters as required for the exchange command (Figure 8-15 of the Aliro spec)

Returns:

bool: True if the counters are valid (< 0xFFFF), else False

decrypt(*ciphertext: bytes, authentication_tag: bytes, ad: bytes = b''*) → bytes

decrypts encrypted data and indicates if verification succeeded

(figure 8-10 of the Aliro spec)

Args:

ciphertext (bytes): Ciphertext. Can be any size. authentication_tag (bytes): Authentication Tag. Must be 16 bytes. ad (bytes, optional): Additional data. Defaults to b''.

Raises:

VerificationError: raised when the verification fails.

Returns:

bytes: decrypted plaintext

encrypt(*data: bytes, ad: bytes = b''*) → tuple[bytes, bytes]

Generates encrypted data + authentication tag

(figure 8-9 of the Aliro spec)

Args:

data (bytes): Data to encrypt. Can be any size. ad (bytes, optional): Additional data. Defaults to b''.

Returns:

tuple[bytes, bytes]: tuple of the encrypted data and the authentication tag.

exception aliro_actuator.access_protocol.encryption.**VerificationError**Bases: [AccessProtocolError](#)

Raised when the verification of an AES decryption fails.

```
aliro_actuator.access_protocol.encryption.create_proprietary_information(type: int, expedited_phase_supported_protocol_versions: list[int], maximum_command_apdu: int | None = None, maximum_response_apdu: int | None = None, vendor_specific_tlv: TLV | None = None) → TLV
```

Creates the proprietary information (see Table 10-2), used for salt generation and Select command.

Args:

type (int): Application type (see Table 10-3) *expedited_phase_supported_protocol_versions* (list[int]): List of supported protocol versions. *maximum_command_apdu* (int | None, optional): Defaults to None. *maximum_response_apdu* (int | None, optional): Defaults to None. *vendor_specific_tlv* (TLV | None, optional): Defaults to None.

Returns:

TLV: Proprietary information (as TLV)

```
aliro_actuator.access_protocol.encryption.create_salt(transport_protocol: TransportProtocol, word: bytes, reader_public_key: PublicKey, reader_ephemeral_public_key: PublicKey, reader_identifier: bytes, protocol_version: bytes, transaction_identifier: bytes, flag: bytes, application_type: int, expedited_phase_supported_protocol_versions: list[int], maximum_command_apdu: int | None = None, maximum_response_apdu: int | None = None, vendor_specific_tlv: TLV | None = None, endpoint_ephemeral_public_key: PublicKey | None = None) → bytes
```

Generates the salt used for key generation

Args:

transport_protocol (TransportProtocol): The transport protocol used. *word* (bytes): “VolatileFast”, “Volatile****” or “Persistent**”. *reader_public_key* (PublicKey). *reader_ephemeral_public_key* (PublicKey). *reader_identifier* (bytes). *protocol_version* (bytes). *transaction_identifier* (bytes). *flag* (bytes): *command_parameters* || *transaction_code*. *application_type* (int): (for proprietary_information generation). *expedited_phase_supported_protocol_versions* (list[int]): (for proprietary_information generation). *maximum_command_apdu* (int | None, optional): (for proprietary_information generation). Defaults to None. *maximum_response_apdu* (int | None, optional): (for proprietary_information generation). Defaults to None. *vendor_specific_tlv* (TLV | None, optional): (for proprietary_information generation). Defaults to None. *endpoint_ephemeral_public_key* (PublicKey | None, optional): only for “VolatileFast” or “Persistent**”. Defaults to None.

Returns:

bytes: the salt as bytes

aliro_actuator.access_protocol.errors module**exception** `aliro_actuator.access_protocol.errors.AccessProtocolError`Bases: `Exception`

Parent class of all errors related to the access protocol.

exception `aliro_actuator.access_protocol.errors.CreateCommandError`Bases: `AccessProtocolError`**exception** `aliro_actuator.access_protocol.errors.CreateResponseError`Bases: `AccessProtocolError`**exception** `aliro_actuator.access_protocol.errors.InvalidAIDError(command: bytes, aid: bytes)`Bases: `InvalidCommandDataError`

Raised when a command with an invalid AID is received.

exception `aliro_actuator.access_protocol.errors.InvalidCLAError(command: bytes)`Bases: `InvalidCommandError`

Raised when the received command has an invalid CLA.

exception `aliro_actuator.access_protocol.errors.InvalidCommandDataError(command: bytes,
message: str | None =
None)`Bases: `InvalidCommandError`

Raised when the received command has an invalid Data field.

exception `aliro_actuator.access_protocol.errors.InvalidCommandError(command: bytes)`Bases: `AccessProtocolError`

Raised when the received command is invalid.

exception `aliro_actuator.access_protocol.errors.InvalidINSError(command: bytes)`Bases: `InvalidCommandError`

Raised when the received command has an invalid INS.

exception `aliro_actuator.access_protocol.errors.InvalidLcError(command: bytes)`Bases: `InvalidCommandError`

Raised when the received command has an invalid LC.

exception `aliro_actuator.access_protocol.errors.InvalidParameterError(command: bytes)`Bases: `InvalidCommandError`

Raised when the received command has an invalid Parameter (P1 or P2).

exception `aliro_actuator.access_protocol.errors.InvalidResponseDataError(response: bytes,
other_info: str = "")`Bases: `InvalidResponseError`

Raised when the received Response has an invalid data.

exception `aliro_actuator.access_protocol.errors.InvalidResponseError(response: bytes)`Bases: `AccessProtocolError`

Raised when the received response is invalid.

exception `aliro_actuator.access_protocol.errors.InvalidStatusError(response: bytes)`

Bases: `InvalidResponseError`

Raised when the received Response has an invalid status.

exception `aliro_actuator.access_protocol.errors.KeyLookupFailed`

Bases: `AccessProtocolError`

exception `aliro_actuator.access_protocol.errors.MessageTooLongError`

Bases: `AccessProtocolError`

Raised when the received command is too long. See 8.3 of the Aliro Spec.

exception `aliro_actuator.access_protocol.errors.SessionError`

Bases: `AccessProtocolError`

exception `aliro_actuator.access_protocol.errors.UnexpectedCommandError`

Bases: `AccessProtocolError`

Raised when a different command is received than expected.

exception `aliro_actuator.access_protocol.errors.UnexpectedResponseError`

Bases: `AccessProtocolError`

Raised when a different Response is received than expected.

exception `aliro_actuator.access_protocol.errors.VersionError`

Bases: `AccessProtocolError`

Raised when the requested version is not supported.

aliro_actuator.access_protocol.mailbox module

class `aliro_actuator.access_protocol.mailbox.Mailbox(initial_data: list[tuple[bytes, int, bytes]] | None = None, size: int = 0, read_permission: bool = True, write_permission: bool = True)`

Bases: `object`

DATA_TAG = 130

INDEX_TAG = 129

START_OF_DATA_TAG = 96

check_boundaries(*offset: int, length: int*) → bool

Checks if the offset and length combination is a valid option for this mailbox.

Args:

`offset (int)`: Offset in bytes. `length (int)`: Data to write.

Returns:

bool: True if offset and length fit in this mailbox, else False

data_is_set() → bool

Checks if some data is different from zeros

Returns:

bool: True if not all data is 0x00, else False

get_data() → bytes

Returns just the mailbox data (no index, tags or lengths).

Returns:

bytes: mailbox data, not including tag (0x82) and length.

get_raw() → bytes

Returns the mailbox contents as bytes.

Returns:

bytes: mailbox contents (including index and data)

get_tlv() → *TLV*

Returns the mailbox contents as a TLV.

Returns:

TLV: Mailbox contents.

read(offset: int, length: int) → bytes

Returns a block of data from the mailbox.

Args:

offset (int): offset in bytes. length (int): length in bytes.

Raises:

MailboxPermissionError: raised if read_permission is not set.

Returns:

bytes: Data from offset to offset + length

set(offset: int, length: int, value: int) → None

Set a block of data to a single value.

Args:

offset (int): offset in bytes. length (int): length in bytes value (int): Every byte from offset to offset + length is set to this value.

Raises:

MailboxPermissionError: raised if write_permission is not set.

write(offset: int, data: bytes) → None

Write a block of data to the mailbox.

Args:

offset (int): Offset in bytes. data (bytes): Data to write.

Raises:

MailboxPermissionError: raised if write_permission is not set.

exception aliro_actuator.access_protocol.mailbox.**MailboxFormatError**

Bases: *AccessProtocolError*

Raised when data passed to the mailbox has the wrong format.

exception aliro_actuator.access_protocol.mailbox.**MailboxPermissionError**

Bases: *AccessProtocolError*

Raised when trying to read/write the mailbox without correct permissions.

aliro_actuator.access_protocol.reader module

```
class aliro_actuator.access_protocol.reader.Reader(transport_protocol: TransportProtocol,
                                                  transport_override: TransportProtocolBase |
                                                  None = None, reader_group_identifier: bytes |
                                                  None = None, reader_group_sub_identifier: bytes
                                                  | None = None, reader_cert: bytes | None = None,
                                                  reader_key: KeyPair | None = None)
```

Bases: *Device*

Simulates a reader device.

Args:

transport_protocol (TransportProtocol): Transport protocol to use. transport_override (TransportProtocolBase | None, optional): Override the transport protocol. Mainly used for testing. Defaults to None. reader_group_identifier (bytes | None, optional): Part of the reader_identifier. Defaults to None. reader_group_sub_identifier (bytes | None, optional): Part of the reader_identifier. Defaults to None. reader_cert (bytes | None, optional): Reader certificate. Defaults to None. reader_key (KeyPair | None, optional): Reader Key. Defaults to None.

Raises:

AccessProtocolError: Raised when arguments have invalid format.

```
command_auth0(transaction: Transaction, transaction_code: TransactionCode, protocol_version: int,
               reader_epubk: bytes, transaction_identifier: bytes, reader_identifier: bytes) → Response
```

Create and send a auth0 command.

Args:

transaction (Transaction): fast or standard transaction_code (TransactionCode): code with instruction (ex. Lock/Unlock) protocol_version (int): reader_epubk (bytes): Reader Ephemeral Key as bytes transaction_identifier (bytes): reader_identifier (bytes):

Returns:

Response: Response containing the received data.

```
command_auth1(expected_response: Auth1Response, request_access_credentials: bool, reader_identifier:
               bytes, endpoint_epubk: PublicKey, reader_epubk: PublicKey, transaction_identifier: bytes,
               encryption: EncryptionEngine | None = None) → Response
```

Create and send a auth1 command.

Args:

expected_response (Auth1Response): key slot or endpoint public key request_access_credentials (bool): request the access credentials if true reader_identifier (bytes): endpoint_epubk (PublicKey): reader_epubk (PublicKey): transaction_identifier (bytes): encryption (EncryptionEngine | None, optional): Encryption engine to decrypt the response. Response will not be decrypted if this is None. Defaults to None.

Returns:

Response: Response containing the received data.

```
command_control_flow(s1: int, s2: int, domain_specific_data: bytes | None = None) → Response
```

Create and send a exchange command.

Args:

s1 (int): s2 (int): domain_specific_data (bytes | None, optional): Defaults to None.

Returns:

Response: Response containing the received data.

command_envelope() → None

command_exchange(*atomic_session: bool, payload: TLV, encryption: EncryptionEngine*) → *Response*

Create and send a exchange command.

Args:

atomic_session (bool): if True, this is part of an atomic session payload (TLV): The payload to send.
encryption (EncryptionEngine): Encryption engine to encrypt the message and decode the response.

Returns:

Response: Response containing the received data.

command_get_response() → None

command_load_cert(*compressed_cert: bytes*) → *Response*

Create and send a load_cert command.

Args:

compressed_cert (bytes): compressed certificate to send.

Returns:

Response: Response containing the received data.

command_select(*aid: bytes*) → *Response*

Create and send a select command.

Args:

aid (bytes): AID to be send.

Returns:

Response: Response containing the received data.

expedited_transaction_fast() → None

expedited_transaction_standard(*transaction_code: TransactionCode, load_cert: bool = False*) → None

Runs the Expedited Standard Phase.

Args:

transaction_code (TransactionCode): Passed during AUTH0. *load_cert* (bool, optional): Runs the load_cert command if True. Defaults to False.

handle_auth0(*transaction_type: Transaction, transaction_code: TransactionCode*) → None

Create and send a AUTH0 command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Args:

transaction_type (Transaction): fast or standard *transaction_code* (TransactionCode): code with instruction (ex. Lock/Unlock)

Raises:

SessionError: Raised if no session is found. AccessProtocolError: Raised if the response has invalid data.

handle_auth1() → None

Create and send a AUTH1 command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Raises:

SessionError: Raised if no session is found.

handle_control_flow(*success: bool*) → None

Create and send a control_flow command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Args:

success (bool): if True, the command will indicate a success status.

Raises:

SessionError: Raised if no session is found.

handle_exchange(*atomic_session: bool, read_requests: list[tuple[int, int]] | None, write_requests: list[tuple[int, bytes]] | None, set_requests: list[tuple[int, int, int]] | None, notify: TLV | None, ursk: bytes | None, update_doc: bytes | None*) → list[bytes]

Create and send a exchange command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Args:

atomic_session (bool): if True, this is part of an atomic session read_requests (list[tuple[int, int]] | None): List of (offset, length) tuples. write_requests (list[tuple[int, bytes]] | None): list of (offset, data) tuples. set_requests (list[tuple[int, int, int]] | None): list of (offset, length, value) tuples. notify (TLV | None): Notify TLV ursk (bytes | None): URSK, for BLE update_doc (bytes | None): request update of an existing Access Document

Raises:

SessionError: Raised if no session is found.

Returns:

list[bytes]: list of read data.

handle_load_cert() → None

Create and send a load_cert command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Raises:

SessionError: Raised if no session is found. AccessProtocolError: Raised if the response has invalid data.

handle_select(*aid: bytes*) → None

create and send a select command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Args:

aid (bytes): AID to be send.

Raises:

SessionError: Raised if no session is found. AccessProtocolError: Raised if the response has invalid data. UnexpectedResponseError: Raised if the response has status/data that cannot be handled

start_new_session(*transaction_identifier: bytes | None = None, ephemeral_key: KeyPair | None = None*) → None

Start a new reader session. Must be done before using handle commands. This sessions stores all information received from commands. Start a new session to delete all received info and start over.

Args:

transaction_identifier (bytes | None, optional): Transaction identifier used for this session. Randomly generated if None. Defaults to None. ephemeral_key (KeyPair | None, optional): ephemeral reader key used for the session. Randomly generated if None. Defaults to None.

step_up_transaction() → None

transaction_initiation() → None

Initializes the hardware and sets up a connection to the card.

class aliro_actuator.access_protocol.reader.**ReaderSession**(*reader_key: KeyPair, reader_identifier: bytes*)

Bases: object

Contains info from a single session (with one User Device)

can_retrieve_access_credential() → bool

can_retrieve_revocation_document() → bool

check_endpoint_authentication() → bool

decrypt_payload(*encrypted_payload: bytes, authentication_tag: bytes*) → bytes

derive_key_volatile(*transport_protocol: TransportProtocol*) → None

encrypt_payload(*payload: bytes*) → tuple[bytes, bytes]

generate_ephemeral_key(*ephemeral_key: KeyPair | None = None*) → None

get_endpoint_ephemeral_key() → bytes

get_reader_epubkey() → *PublicKey*

set_auth1_info(*auth1_response: Response, expected_response: Auth1Response*) → None

set_endpoint_ephemeral_key(*key: bytes*) → None

set_flag(*transaction: Transaction, transaction_code: TransactionCode*) → None

set_select_info(*select_response: Response*) → None

set_shared_key() → None

step_up_aid_select_required() → bool

property transaction_identifier: bytes

aliro_actuator.access_protocol.tlv module

class aliro_actuator.access_protocol.tlv.**TLV**(*data: list[tuple[int, bytes | list]]*)

Bases: object

Class to handle TLV structures (BER-TLV, ISO 7816-4) (use the from_bytes method to create this struct from a bytestring)

Args:

data (list[tuple[int, bytes | list]]): list of tuples. The tuples consist of tags and values. The values can be either bytes or a list of tag/value tuples. (length is calculated using the data and does not need to be specified)

add_value(*tag: int, data: bytes*) → None

add a new tag/value pair to the TLV

Args:

tag (int): tag *data* (bytes): value

static from_bytes(*data_bytes: bytes*) → *TLV*

Converts tlv value (BER-TLV, ISO 7816-4) from bytestring to TLV class Value is b"" if the tlv tag has no value. (length is not represented in the class, but can be derived from the value)

get_all_bytes_of_tag(*tag: int*) → list[bytes]

Similar to get_all_of_tag, but the list only contains bytes objects. Raises an error when a no byte object is found.

Args:

tag (int): tag number

Raises:

TlvError: Raised when value is not of type bytes.

Returns:

list[bytes]: list with all values with given tag

get_all_of_tag(*tag: int*) → list[bytes | *TLV*]

returns a list with all values with a given tag.

Args:

tag (int): tag number

Returns:

list[bytes | TLV]: list with all values with given tag. Can be empty if tag was not found.

get_bytes(*tag: int, index: int = 0*) → bytes

Similar to get_value, but always returns a bytestring. Raises error when requested element is not a bytestring

Args:

tag (int): tag number index (int, optional): when there are multiple elements with the same tag, use this to differentiate them. Defaults to 0.

Raises:

IndexError: Raised when a tag is not found. TlvError: Raised when value is not of type bytes.

Returns:

bytes: value

get_tlv(*tag: int, index: int = 0*) → *TLV*

Similar to get_value, but always returns a TLV object. Raises error when requested element is not a TLV object

Args:

tag (int): tag number index (int, optional): when there are multiple elements with the same tag, use this to differentiate them. Defaults to 0.

Raises:

IndexError: Raised when a tag is not found. TlvError: Raised when value is not of type TLV.

Returns:

TLV: value

get_value(*tag: int, index: int = 0*) → bytes | *TLV*

Get the value of a tag. use index if there are multiple tags with the same name

Args:

tag (int): tag to find. index (int, optional): use to differentiate multiple values with the same tag. Defaults to 0.

Raises:

IndexError: Raised when a tag is not found.

Returns:

bytes | TLV: the value as bytes or TLV. Type depends on the structure of the TLV.

to_bytes() → bytes

Converts tlv value (BER-TLV, ISO 7816-4) to bytestring. (length is derived from the value)

to_data() → list

returns the tlv as a list of tuples

exception `aliro_actuator.access_protocol.tlv.TlvError`

Bases: [AccessProtocolError](#)

Raised when a TLV is malformed, has unexpected items or is missing items.

aliro_actuator.access_protocol.user_device module

class `aliro_actuator.access_protocol.user_device.MailboxSession`

Bases: object

class `Command(value)`

Bases: Enum

An enumeration.

SET = 0

WRITE = 1

add_set(data: bytes | None) → None

add_write(data: bytes | None) → None

execute_commands(mailbox: Mailbox) → None

is_started() → bool

start() → None

stop() → None

class `aliro_actuator.access_protocol.user_device.UserDevice`(transport_protocol:

[TransportProtocol](#),

transport_override:

[TransportProtocolBase](#) | None =

None, endpoints: list[[Endpoint](#)] = [],

supported_versions: list[int] = [256],

mailbox: int | list[tuple[bytes, int,

bytes]] | None = None, mailbox_read:

bool = True, mailbox_write: bool =

True)

Bases: [Device](#)

Simulates a user device.

Args:

transport_protocol ([TransportProtocol](#)): The transport protocol to use. endpoints (list[bytes], optional): list of endpoints. Defaults to []. supported_versions (list[int], optional): List of supported protocol versions. Defaults to [PROTOCOL_VERSION]. mailbox (int | list[tuple[bytes, int, bytes]] | None): If None, don't

use mailbox. If int, use mailbox with this size in bytes. If list, use mailbox with this initial data. List should consist of OUI, type, data tuples.

handle_auth0(*auth0_command*: [Command](#)) → None

Parse a auth0 command and send the appropriate response.

Args:

auth0_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state. VersionError: Raised when the protocol version is not supported. NotImplementedError:

handle_auth1(*auth1_command*: [Command](#)) → None

Parse a auth1 command and send the appropriate response.

Args:

auth1_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state. KeyLookupFailed: When the reader public key cannot be found. AccessProtocolError: Raised if the response has invalid data.

handle_control_flow(*control_flow_command*: [Command](#)) → None

Parse an control flow command and send the appropriate response.

Args:

control_flow_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state.

handle_exchange(*control_flow_command*: [Command](#)) → None

Parse an exchange command and send the appropriate response.

Args:

control_flow_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state. AccessProtocolError: Raised if the response has invalid data.

handle_load_cert(*load_cert_command*: [Command](#)) → None

Parse a load cert command and send the appropriate response.

Args:

load_cert_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state. KeyLookupFailed: When the reader public key cannot be found.

handle_select(*select_command*: [Command](#)) → None

Parse a select command and send the appropriate response.

Args:

select_command (Command): The command to respond to.

Raises:

SessionError: Raised if no session is found. InvalidAIDError: Raised when the AID is invalid.

main_loop() → None

Starts a loop, where every command received is replied with an appropriate response. Should keep running, even when receiving invalid commands.

Raises:

SessionError: When starting a new session failed. NotImplementedError: When a command which is not implemented is received.

response_auth0(*endpoint_epubk: bytes, cryptogram: bytes | None = None*) → None

Create and send an auth0 response.

Args:

endpoint_epubk (bytes): Endpoint Ephemeral public key. cryptogram (bytes | None, optional): authentication cryptogram. Defaults to None.

response_auth1(*key_slot: bytes | None, endpoint_public_key: bytes | None, expected_response:*

Auth1Response, signature: bytes, encryption: EncryptionEngine, status: int = StatusBytes.SUCCESS, private_mailbox_data: bytes | None = None, access_doc_retrieve: bool = False, revocation_doc_retrieve: bool = False, step_up_aid_required: bool = False, data_in_mailbox: bool = False, read_mailbox: bool = False, write_mailbox: bool = False, send_issuer_backend: bool = False, send_bound_app: bool = False, update_doc: bool = False, credential_signed_timestamp: bytes | None = None, revocation_signed_timestamp: bytes | None = None) → None

Create and send an auth1 response.

Args:

key_slot (bytes | None): First 8 bytes of the keyIdentifier. endpoint_public_key (bytes | None): Endpoint long term public key. expected_response (Auth1Response): expected response (keyslot or endpoint public key) signature (bytes): Endpoint authentication signature. encryption (EncryptionEngine): Encryption engine to encrypt the response. status (int, optional): response status. Defaults to StatusBytes.SUCCESS. private_mailbox_data (bytes | None, optional): Defaults to None. access_doc_retrieve (bool, optional): Part of signaling bitmap. Defaults to False. revocation_doc_retrieve (bool, optional): Part of signaling bitmap. Defaults to False. step_up_aid_required (bool, optional): Part of signaling bitmap. Defaults to False. data_in_mailbox (bool, optional): Part of signaling bitmap. Defaults to False. read_mailbox (bool, optional): Part of signaling bitmap. Defaults to False. write_mailbox (bool, optional): Part of signaling bitmap. Defaults to False. send_issuer_backend (bool, optional): Part of signaling bitmap. Defaults to False. send_bound_app (bool, optional): Part of signaling bitmap. Defaults to False. update_doc (bool, optional): Part of signaling bitmap. Defaults to False. credential_signed_timestamp (bytes | None, optional): Defaults to None. revocation_signed_timestamp (bytes | None, optional): Defaults to None.

response_control_flow() → None

Create and send a control flow response.

response_envelope() → None

response_error(*status_bytes: int*) → None

Create and send an error response.

response_exchange(*payload: bytes, encryption: EncryptionEngine*) → None

Create and send a exchange response.

Args:

payload (bytes): exchange response payload. encryption (EncryptionEngine): encryption engine to encrypt the payload.

response_get_response() → None

response_load_cert() → None

Create and send a load cert response.

response_select(*aid: bytes, type: int, protocol_versions: list[int], maximum_command_apdu: int | None = None, maximum_response_apdu: int | None = None, vendor_specific_tlv: TLV | None = None*) → None

Create and send a select response.

Args:

aid (bytes): AID type (*int*): Application type (table 10-4) *protocol_versions* (list[int]): Expedited phase supported protocol versions. *maximum_command_apdu* (int | None, optional): Defaults to None. *maximum_response_apdu* (int | None, optional): Defaults to None. *vendor_specific_tlv* (TLV | None, optional): Defaults to None.

return_exchange_error_and_close_channel() → None

Return an exchange error and close the channel. Used when an exchange fails.

Raises:

SessionError: Raised if no session is found. *AccessProtocolError*: Raised if no encryption engine is found.

start_new_session(*ephemeral_key: KeyPair | None = None*) → None

Start a new user session. Must be done before using handle commands. This sessions stores all information received from commands. Start a new session to delete all received info and start over.

Args:

ephemeral_key (KeyPair | None, optional): ephemeral reader key used for the session. Randomly generated if None. Defaults to None.

transaction_initiation() → None

Initializes the hardware and sets up a connection to the reader.

wait_for_command(*expected_command: INS | list[INS] | None = None, encryption: EncryptionEngine | None = None*) → *Command*

Waits until a command is received, and parses the command.

Args:

expected_command (INS | list[INS] | None, optional): INS or list of INS with expected commands. raises *UnexpectedCommandError* if another command is received. Defaults to None. *encryption* (EncryptionEngine | None, optional): Used for decrypting messages. Not required for every command. Defaults to None.

Raises:

InvalidCLAError: Raised when the received command has an invalid CLA. *InvalidParameterError*: Raised when the received command has an invalid Parameter (P1 or P2). *InvalidCommandError*: Raised when the received command is invalid. *VerificationError*: Raised when the verification of an AES decryption fails. *UnexpectedCommandError*: when the command is not in *expected_command*.

Returns:

Command: the received command.

class aliro_actuator.access_protocol.user_device.**UserSession**(*supported_version: list[int]*)

Bases: object

Contains info from a single session (with one Reader Device)

derive_key_persistent(*transport_protocol: TransportProtocol*) → None

```

derive_key_volatile(transport_protocol: TransportProtocol) → None
generate_ephemeral_key(ephemeral_key: KeyPair | None = None) → None
get_endpoint_epubkey() → PublicKey
get_endpoint_reader_public_key(endpoints: list[Endpoint]) → PublicKey | None
get_reader_public_key() → PublicKey
get_transaction_type() → Transaction
set_auth0_data(auth0_command: Command) → None
set_cert_and_verify(compressed_cert: bytes, public_key: PublicKey) → bool
set_endpoint(endpoint: Endpoint) → None
set_reader_public_key(endpoints: list[Endpoint]) → None
set_shared_key() → None
state_valid(state: list[UserSessionState] | UserSessionState) → bool
update_state(state: UserSessionState) → None

```

```

class aliro_actuator.access_protocol.user_device.UserSessionState(value)

```

Bases: Enum

An enumeration.

```

AUTH0_FAST_DONE = 4

```

```

AUTH0_STD_DONE = 3

```

```

AUTH1_DONE = 5

```

```

EXCHANGE_DONE = 6

```

```

SELECT_DONE = 2

```

```

SESSION_START = 1

```

Module contents

Access Protocol

Implements the access protocol part of the aliro protocol

```

class aliro_actuator.access_protocol.Device(transport_protocol: TransportProtocol, transport_override:
                                           TransportProtocolBase | None = None)

```

Bases: object

```

property support_extended_length_apdu: bool

```

aliro_actuator.credential package

Submodules

aliro_actuator.credential.access_credential module

class aliro_actuator.credential.access_credential.**AccessCredential**

Bases: *Credential*

parse() → None

validate_credential() → None

class aliro_actuator.credential.access_credential.**DataElement**(*id: bytes | None, schedules: list | None, access_rules: list | None, reader_rules_ids: list | None, validity_iteration: int | None*)

Bases: object

class aliro_actuator.credential.access_credential.**IssuerAuth**

Bases: object

aliro_actuator.credential.errors module

exception aliro_actuator.credential.errors.**CredentialError**

Bases: Exception

exception aliro_actuator.credential.errors.**ElementSizeError**(*actual: int, maximum: int | None = None, expected: int | None = None, minimum: int | None = None*)

Bases: Exception

aliro_actuator.credential.revocation_document module

class aliro_actuator.credential.revocation_document.**RevocationDocument**

Bases: *Credential*

Module contents

class aliro_actuator.credential.**Credential**

Bases: object

retrieve() → None

store() → None

aliro_actuator.hw_driver package

Subpackages

aliro_actuator.hw_driver.pn7160_driver package

Submodules

aliro_actuator.hw_driver.pn7160_driver.api module

class aliro_actuator.hw_driver.pn7160_driver.api.**TECHNOLOGY_MASK**(*value*)

Bases: IntEnum

An enumeration.

DEFAULT = -1

MASK_A = 1

class aliro_actuator.hw_driver.pn7160_driver.api.**ndef_info_t**

Bases: Structure

current_ndef_length

Structure/Union member

is_def

Structure/Union member

is_writable

Structure/Union member

max_ndef_length

Structure/Union member

class aliro_actuator.hw_driver.pn7160_driver.api.**nfcHostCardEmulationCallback_t**

Bases: Structure

onDataReceived

Structure/Union member

onHostCardEmulationActivated

Structure/Union member

onHostCardEmulationDeactivated

Structure/Union member

class aliro_actuator.hw_driver.pn7160_driver.api.**nfcTagCallback_t**

Bases: Structure

onTagArrival

Structure/Union member

onTagDeparture

Structure/Union member

```
class aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t
```

```
    Bases: Structure
```

```
    handle
```

```
        Structure/Union member
```

```
    protocol
```

```
        Structure/Union member
```

```
    technology
```

```
        Structure/Union member
```

```
    uid
```

```
        Structure/Union member
```

```
    uid_length
```

```
        Structure/Union member
```

```
aliro_actuator.hw_driver.pn7160_driver.errors module
```

```
exception aliro_actuator.hw_driver.pn7160_driver.errors.DriverNotInitializedError(error_message:  
                                                                                   str | None =  
                                                                                   None)
```

```
    Bases: PN7160DriverError
```

```
exception aliro_actuator.hw_driver.pn7160_driver.errors.NCIErrror(error_code: int,  
                                                                    error_message: str | None =  
                                                                    None)
```

```
    Bases: PN7160DriverError
```

```
exception aliro_actuator.hw_driver.pn7160_driver.errors.NCINotFoundError
```

```
    Bases: PN7160DriverError
```

```
exception aliro_actuator.hw_driver.pn7160_driver.errors.NoDataReceivedError
```

```
    Bases: PN7160DriverError
```

```
exception aliro_actuator.hw_driver.pn7160_driver.errors.NoReaderError(error_message: str |  
                                                                           None = None)
```

```
    Bases: PN7160DriverError
```

```
exception aliro_actuator.hw_driver.pn7160_driver.errors.NoTagError(error_message: str | None =  
                                                                           None)
```

```
    Bases: PN7160DriverError
```

```
exception aliro_actuator.hw_driver.pn7160_driver.errors.PN7160DriverError
```

```
    Bases: Exception
```

Module contents

```
class aliro_actuator.hw_driver.pn7160_driver.Driver(nci_location: str | None = None)
    Bases: object
    initialize(mode: Mode) → None
    receive_message() → bytes
    send_message(message: bytes) → None
    wait_for_reader() → None
    wait_for_tag() → None
```

Module contents

aliro_actuator.transport_protocol package

Submodules

aliro_actuator.transport_protocol.ble_uwb module

```
class aliro_actuator.transport_protocol.ble_uwb.BLEUWB
    Bases: TransportProtocolBase
    get_message() → bytes
    initialization(mode: Mode) → None
    send_message(command: bytes) → None
    wait_for_connection() → None
```

aliro_actuator.transport_protocol.errors module

```
exception aliro_actuator.transport_protocol.errors.InvalidModeError
    Bases: TransportProtocolError
exception aliro_actuator.transport_protocol.errors.NoDataReceivedError
    Bases: TransportProtocolError
exception aliro_actuator.transport_protocol.errors.TransportProtocolError
    Bases: Exception
```

aliro_actuator.transport_protocol.nfc module

```
class aliro_actuator.transport_protocol.nfc.NFC(port: str | None = None)
    Bases: TransportProtocolBase
    get_message() → bytes
    initialization(mode: Mode) → None
    send_message(command: bytes) → None
    wait_for_connection() → None
```

aliro_actuator.transport_protocol.socket module

```
class aliro_actuator.transport_protocol.socket.Socket
    Bases: TransportProtocolBase
    Uses sockets to communicate. Mainly used for testing purposes.
    disconnect() → None
    get_message() → bytes
    initialization(mode: Mode) → None
    send_message(command: bytes) → None
    wait_for_connection() → None
```

Module contents

```
class aliro_actuator.transport_protocol.Mode(value)
    Bases: Enum
    An enumeration.
    CARD_EMULATION = 2
    READER = 1

class aliro_actuator.transport_protocol.TransportProtocolBase
    Bases: ABC
    abstract get_message() → bytes
    abstract initialization(mode: Mode) → None
    abstract send_message(command: bytes) → None
    abstract wait_for_connection() → None
```

aliro_actuator.trust_framework package**Submodules****aliro_actuator.trust_framework.certificate module**

```
class aliro_actuator.trust_framework.certificate.Certificate(version: bytes = b", serial_number:
    bytes = b", signature: bytes = b",
    issuer: bytes = b",
    validity_not_before: bytes = b",
    validity_not_after: bytes = b",
    subject: bytes = b",
    key_info_algorithm: bytes = b",
    key_info_parameters: bytes = b",
    key_info_subject_public_key: bytes
    = b", authority_key_identifier: bytes
    = b", key_usage_extension: bytes =
    b", signature_algorithm: bytes = b",
    signature_value: bytes = b")
```

Bases: object

classmethod **decode**(certificate: bytes) → *Certificate*

classmethod **decode_compressed**(compressed_certificate: bytes) → *Certificate*

default_issuer = b'issuer'

default_serial_number = b'\x01'

default_subject = b'subject'

default_validity_not_after = b'490101000000Z'

default_validity_not_before = b'200101000000Z'

encode_compressed() → bytes

get_public_key() → *PublicKey*

verify(key: *PublicKey*, data: bytes) → bool

aliro_actuator.trust_framework.endpoint module

```
class aliro_actuator.trust_framework.endpoint.Endpoint(user_device_key_pair: KeyPair,
    reader_public_key: PublicKey,
    reader_identifier: list[bytes], key_slot: bytes
    = b")
```

Bases: object

get_endpoint_public_key() → *PublicKey*

get_reader_public_key() → *PublicKey*

has_identifier(identifier: bytes) → bool

sign(data: bytes) → bytes

aliro_actuator.trust_framework.errors module

exception `aliro_actuator.trust_framework.errors.CertificateDecodingError`(*certificate: bytes, message: str | None = None*)

Bases: `TrustFrameworkError`

Passed when a certificate cannot be decoded.

exception `aliro_actuator.trust_framework.errors.InvalidKeyError`(*key: bytes, message: str | None = None*)

Bases: `TrustFrameworkError`

Passed when a key is invalid.

exception `aliro_actuator.trust_framework.errors.InvalidKeyFormatError`

Bases: `TrustFrameworkError`

Passed when a key has the wrong format.

exception `aliro_actuator.trust_framework.errors.TrustFrameworkError`

Bases: `Exception`

Parent class of all errors related to the trust framework.

aliro_actuator.trust_framework.key module

class `aliro_actuator.trust_framework.key.Key`

Bases: `object`

Base class for all keys.

class `aliro_actuator.trust_framework.key.KeyPair`(*private_key: str | None = None, public_key: bytes | str | None = None*)

Bases: `object`

Combination of a private and public key.

get_private_key() → `PrivateKey`

Returns the private key.

get_public_key() → `PublicKey`

Returns the public key.

get_public_key_as_bytes() → `bytes`

Returns the public key as raw bytes. Prepend with 0x04, total length of 65.

sign(data: bytes) → `bytes`

Sign the data. The signature is returned.

(figure 8-5 of Aliro spec)

verify(data: bytes, signature: bytes) → `bool`

Verifies the data. Returns True when the verification succeeds.

(figure 8-6 of Aliro spec)

class aliro_actuator.trust_framework.key.**PrivateKey**(key: str | None = None)

Bases: [Key](#)

Private Key

as_pem() → str

Returns the key as PEM.

compute_shared_key(public_key: [PublicKey](#), shared_info: bytes) → bytes

Compute a shared key using Diffie-Hellman

(figure 8-7 of the Aliro spec)

generate_public_key() → [PublicKey](#)

Generate a public key from this private key.

sign(data: bytes) → bytes

Sign the data. The signature is returned.

(figure 8-5 of Aliro spec)

class aliro_actuator.trust_framework.key.**PublicKey**(key: bytes | *EllipticCurvePublicKey* | str)

Bases: [Key](#)

Public Key

as_bytes() → bytes

Returns the key as raw bytes. Prepend with 0x04, total length of 65.

as_pem() → str

Returns the key as PEM.

get_x() → int

get_y() → int

verify(data: bytes, signature: bytes) → bool

Verifies the data. Returns True when the verification succeeds.

(figure 8-6 of Aliro spec)

aliro_actuator.trust_framework.key.**derive_key**(input_key: bytes, info: bytes, length: int, salt: bytes) → bytes

Derive key

(figure 8-8 of the Aliro spec)

Module contents

3.1.1.2 Submodules

aliro_actuator.log module

class aliro_actuator.log.**InterceptHandler**(log_source: str)

Bases: [Handler](#)

This class is used to intercept the standard python logging. To enable it, you must call.

This is based on boilerplate code from the loguru project

```
logging.basicConfig(handlers=[InterceptHandler()], level=0)
```

```
emit(record: LogRecord) → None
```

This will be called by the standard logging module on each log call

Args:

record (logging.LogRecord): The log record intercepted from the standard logging module

```
loglevel_mapping = {0: 'NOTSET', 10: 'DEBUG', 20: 'INFO', 30: 'WARNING', 40: 'ERROR', 50: 'CRITICAL'}
```

```
exception aliro_actuator.log.LoggingConfigError
```

Bases: Exception

Raised when config parameters are missing or don't align

```
aliro_actuator.log.configure_logging() → None
```

3.1.1.3 Module contents

```
class aliro_actuator.Global
```

Bases: object

```
logger = <loguru.logger handlers=[(id=1, level=10, sink=<stdout>), (id=2, level=10, sink='/app/src/./logs/{time:YYYY-MM-DD--HH-mm-ss}.log')]]>
```

```
classmethod set_logger(logger) → None
```


INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

a

- `aliro_actuator`, 39
- `aliro_actuator.access_protocol`, 30
- `aliro_actuator.access_protocol.apdu`, 7
- `aliro_actuator.access_protocol.authentication`, 13
- `aliro_actuator.access_protocol.defines`, 13
- `aliro_actuator.access_protocol.encryption`, 16
- `aliro_actuator.access_protocol.errors`, 18
- `aliro_actuator.access_protocol.mailbox`, 19
- `aliro_actuator.access_protocol.reader`, 21
- `aliro_actuator.access_protocol.tlv`, 24
- `aliro_actuator.access_protocol.user_device`, 26
- `aliro_actuator.credential`, 31
- `aliro_actuator.credential.access_credential`, 31
- `aliro_actuator.credential.errors`, 31
- `aliro_actuator.credential.revocation_document`, 31
- `aliro_actuator.hw_driver`, 34
- `aliro_actuator.hw_driver.pn7160_driver`, 34
- `aliro_actuator.hw_driver.pn7160_driver.api`, 32
- `aliro_actuator.hw_driver.pn7160_driver.errors`, 33
- `aliro_actuator.log`, 38
- `aliro_actuator.transport_protocol`, 35
- `aliro_actuator.transport_protocol.ble_uwb`, 34
- `aliro_actuator.transport_protocol.errors`, 34
- `aliro_actuator.transport_protocol.nfc`, 35
- `aliro_actuator.transport_protocol.socket`, 35
- `aliro_actuator.trust_framework`, 38
- `aliro_actuator.trust_framework.certificate`, 36
- `aliro_actuator.trust_framework.endpoint`, 36
- `aliro_actuator.trust_framework.errors`, 37
- `aliro_actuator.trust_framework.key`, 37

INDEX

A

ACCESS_RESPONSE_TAG
 (*aliro_actuator.access_protocol.defines.Auth1*
 attribute), 13
 AccessCredential (class in
 aliro_actuator.credential.access_credential),
 31
 AccessProtocolError, 18
 add_set() (*aliro_actuator.access_protocol.user_device.MailboxSession*
 method), 26
 add_value() (*aliro_actuator.access_protocol.tlv.TLV*
 method), 24
 add_write() (*aliro_actuator.access_protocol.user_device.MailboxSession*
 method), 26
 AID_TAG (*aliro_actuator.access_protocol.defines.Select*
 attribute), 15
 aliro_actuator
 module, 39
 aliro_actuator.access_protocol
 module, 30
 aliro_actuator.access_protocol.apdu
 module, 7
 aliro_actuator.access_protocol.authentication
 module, 13
 aliro_actuator.access_protocol.defines
 module, 13
 aliro_actuator.access_protocol.encryption
 module, 16
 aliro_actuator.access_protocol.errors
 module, 18
 aliro_actuator.access_protocol.mailbox
 module, 19
 aliro_actuator.access_protocol.reader
 module, 21
 aliro_actuator.access_protocol.tlv
 module, 24
 aliro_actuator.access_protocol.user_device
 module, 26
 aliro_actuator.credential
 module, 31
 aliro_actuator.credential.access_credential
 module, 31
 aliro_actuator.credential.errors
 module, 31
 aliro_actuator.credential.revocation_document
 module, 31
 aliro_actuator.hw_driver
 module, 34
 aliro_actuator.hw_driver.pn7160_driver
 module, 34
 aliro_actuator.hw_driver.pn7160_driver.api
 module, 32
 aliro_actuator.hw_driver.pn7160_driver.errors
 module, 33
 aliro_actuator.log
 module, 38
 aliro_actuator.transport_protocol
 module, 35
 aliro_actuator.transport_protocol.ble_uwb
 module, 34
 aliro_actuator.transport_protocol.errors
 module, 34
 aliro_actuator.transport_protocol.nfc
 module, 35
 aliro_actuator.transport_protocol.socket
 module, 35
 aliro_actuator.trust_framework
 module, 38
 aliro_actuator.trust_framework.certificate
 module, 36
 aliro_actuator.trust_framework.endpoint
 module, 36
 aliro_actuator.trust_framework.errors
 module, 37
 aliro_actuator.trust_framework.key
 module, 37
 APDU (class in *aliro_actuator.access_protocol.apdu*), 7
 as_bytes() (*aliro_actuator.trust_framework.key.PublicKey*
 method), 38
 as_pem() (*aliro_actuator.trust_framework.key.PrivateKey*
 method), 38
 as_pem() (*aliro_actuator.trust_framework.key.PublicKey*
 method), 38
 AUTH0 (*aliro_actuator.access_protocol.apdu.INS* at-

tribute), 10

Auth0 (class in aliro_actuator.access_protocol.defines), 13

AUTH0_FAST_DONE (aliro_actuator.access_protocol.user_device.UserSessionState attribute), 30

AUTH0_STD_DONE (aliro_actuator.access_protocol.user_device.UserSessionState attribute), 30

AUTH1 (aliro_actuator.access_protocol.apdu.INS attribute), 10

Auth1 (class in aliro_actuator.access_protocol.defines), 13

AUTH1_DONE (aliro_actuator.access_protocol.user_device.UserSessionState attribute), 30

Auth1Response (class in aliro_actuator.access_protocol.apdu), 8

B

BLE_UWB (aliro_actuator.access_protocol.defines.TransportProtocol attribute), 15

BLEUWB (class in aliro_actuator.transport_protocol.ble_uwb), 34

C

can_retrieve_access_credential() (aliro_actuator.access_protocol.reader.ReaderSession method), 24

can_retrieve_revocation_document() (aliro_actuator.access_protocol.reader.ReaderSession method), 24

CARD_EMULATION (aliro_actuator.transport_protocol.Mode attribute), 35

Certificate (class in aliro_actuator.trust_framework.certificate), 36

CERTIFICATE_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 13

CertificateDecodingError, 37

check_boundaries() (aliro_actuator.access_protocol.math_box.MathBox method), 19

check_counters_valid() (aliro_actuator.access_protocol.encryption.EncryptionEngine method), 16

check_endpoint_authentication() (aliro_actuator.access_protocol.reader.ReaderSession method), 24

Command (class in aliro_actuator.access_protocol.apdu), 9

command_auth0() (aliro_actuator.access_protocol.reader.Reader method), 21

command_auth1() (aliro_actuator.access_protocol.reader.Reader method), 21

command_control_flow() (aliro_actuator.access_protocol.reader.Reader method), 21

command_envelope() (aliro_actuator.access_protocol.reader.Reader method), 21

command_exchange() (aliro_actuator.access_protocol.reader.Reader method), 22

command_get_response() (aliro_actuator.access_protocol.reader.Reader method), 22

command_load_cert() (aliro_actuator.access_protocol.reader.Reader method), 22

COMMAND_NOT_ALLOWED

COMMAND_NOT_COMPLIANT (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 11

command_select() (aliro_actuator.access_protocol.reader.Reader method), 22

COMMAND_TAG (aliro_actuator.access_protocol.defines.Auth0 attribute), 13

COMMAND_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 13

compute_shared_key() (aliro_actuator.trust_framework.key.PrivateKey method), 38

CONDITIONS_OF_USE_NOT_SATISFIED (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 11

configure_logging() (in module aliro_actuator.log), 39

CONTROL_FLOW (aliro_actuator.access_protocol.apdu.INS attribute), 10

ControlFlow (class in aliro_actuator.access_protocol.defines), 14

create_auth0_command() (aliro_actuator.access_protocol.apdu.APDU method), 7

create_auth0_response() (aliro_actuator.access_protocol.apdu.APDU method), 7

create_auth1_command() (aliro_actuator.access_protocol.apdu.APDU method), 7

create_auth1_response() (aliro_actuator.access_protocol.apdu.APDU method), 7

create_command() (aliro_actuator.access_protocol.apdu.APDU method), 8

create_control_flow_command() (aliro_actuator.access_protocol.apdu.APDU method), 8

create_control_flow_response() (aliro_actuator.access_protocol.apdu.APDU method), 8

`create_endpoint_authentication()` (in module `aliro_actuator.access_protocol.authentication`), 13
`create_envelope_command()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_envelope_response()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_error_response()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_exchange_command()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_exchange_response()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_from_bytestring()` (`aliro_actuator.access_protocol.apdu.Command` class method), 9
`create_from_bytestring()` (`aliro_actuator.access_protocol.apdu.Response` class method), 10
`create_from_parameters()` (`aliro_actuator.access_protocol.apdu.Command` class method), 9
`create_from_parameters()` (`aliro_actuator.access_protocol.apdu.Response` class method), 10
`create_get_response_command()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_get_response_response()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_load_cert_command()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_load_cert_response()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_proprietary_information()` (in module `aliro_actuator.access_protocol.encryption`), 16
`create_reader_authentication()` (in module `aliro_actuator.access_protocol.authentication`), 13
`create_salt()` (in module `aliro_actuator.access_protocol.encryption`), 17
`create_select_command()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`create_select_response()` (`aliro_actuator.access_protocol.apdu.APDU` method), 8
`CreateCommandError`, 18
`CreateResponseError`, 18
`Credential` (class in `aliro_actuator.credential`), 31
`CREDENTIAL_TIMESTAMP_LEN` (`aliro_actuator.access_protocol.defines.Auth1` attribute), 13
`CREDENTIAL_TIMESTAMP_TAG` (`aliro_actuator.access_protocol.defines.Auth1` attribute), 13
`CredentialError`, 31
`CRYPTOGRAM_TAG` (`aliro_actuator.access_protocol.defines.Auth0` attribute), 13
`current_ndef_length` (`aliro_actuator.hw_driver.pn7160_driver.api.ndef_info_t` attribute), 32
D
`data_is_set()` (`aliro_actuator.access_protocol.mailbox.Mailbox` method), 19
`DATA_TAG` (`aliro_actuator.access_protocol.mailbox.Mailbox` attribute), 19
`DataElement` (class in `aliro_actuator.credential.access_credential`), 31
`decode()` (`aliro_actuator.trust_framework.certificate.Certificate` class method), 36
`decode_compressed()` (`aliro_actuator.trust_framework.certificate.Certificate` class method), 36
`decrypt()` (`aliro_actuator.access_protocol.encryption.EncryptionEngine` method), 16
`decrypt_payload()` (`aliro_actuator.access_protocol.reader.ReaderSession` method), 24
`DEFAULT` (`aliro_actuator.hw_driver.pn7160_driver.api.TECHNOLOGY_MA` attribute), 32
`default_issuer` (`aliro_actuator.trust_framework.certificate.Certificate` attribute), 36
`default_serial_number` (`aliro_actuator.trust_framework.certificate.Certificate` attribute), 36
`default_subject` (`aliro_actuator.trust_framework.certificate.Certificate` attribute), 36
`default_validity_not_after` (`aliro_actuator.trust_framework.certificate.Certificate` attribute), 36
`default_validity_not_before` (`aliro_actuator.trust_framework.certificate.Certificate` attribute), 36
`derive_key()` (in module `aliro_actuator.trust_framework.key`), 38
`derive_key_persistent()` (`aliro_actuator.access_protocol.user_device.UserSession` method), 29

derive_key_volatile() (aliro_actuator.access_protocol.reader.ReaderSession method), 24
 derive_key_volatile() (aliro_actuator.access_protocol.user_device.UserSession method), 29
 Device (class in aliro_actuator.access_protocol), 30
 DeviceType (class in aliro_actuator.access_protocol.encryption), 16
 DISARM (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 12
 DISARM_WITH_FORCE_USER_AUTHENTICATION (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 12
 disconnect() (aliro_actuator.transport_protocol.socket.Socket method), 35
 DOMAIN_SPECIFIC_TAG (aliro_actuator.access_protocol.defines.ControlFlow attribute), 14
 Driver (class in aliro_actuator.hw_driver.pn7160_driver), 34
 DriverNotInitializedError, 33
E
 ElementSizeError, 31
 emit() (aliro_actuator.log.InterceptHandler method), 39
 encode_compressed() (aliro_actuator.trust_framework.certificate.Certificate method), 36
 encrypt() (aliro_actuator.access_protocol.encryption.EncryptionEngine method), 16
 encrypt_payload() (aliro_actuator.access_protocol.reader.ReaderSession method), 24
 EncryptionEngine (class in aliro_actuator.access_protocol.encryption), 16
 Endpoint (class in aliro_actuator.trust_framework.endpoint), 36
 ENDPOINT_EPUBK_LEN (aliro_actuator.access_protocol.defines.Auth1 attribute), 14
 ENDPOINT_EPUBK_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 13
 ENDPOINT_EPUBK_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 14
 ENDPOINT_EPUBK_TAG (aliro_actuator.access_protocol.defines.Endpoint attribute), 14
 ENDPOINT_EPUBK_TAG (aliro_actuator.access_protocol.defines.Endpoint attribute), 15
 ENDPOINT_PUBLIC_KEY (aliro_actuator.access_protocol.apdu.Auth1Response attribute), 8
 ENDPOINT_SIG_LEN (aliro_actuator.access_protocol.defines.Auth1 attribute), 14
 ENDPOINT_SIG_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 14
 EndpointAuth (class in aliro_actuator.access_protocol.defines), 14
 ENVELOPE (aliro_actuator.access_protocol.apdu.INS attribute), 10
 ETSPV_TAG (aliro_actuator.access_protocol.defines.Auth0 attribute), 13
 ETSPV_TAG (aliro_actuator.access_protocol.defines.Select attribute), 15
 EXCHANGE (aliro_actuator.access_protocol.apdu.INS attribute), 10
 Exchange (class in aliro_actuator.access_protocol.defines), 14
 EXCHANGE_DONE (aliro_actuator.access_protocol.user_device.UserSession attribute), 30
 execute_commands() (aliro_actuator.access_protocol.user_device.Mailbox method), 26
 expedited_transaction_fast() (aliro_actuator.access_protocol.reader.Reader method), 22
 expedited_transaction_standard() (aliro_actuator.access_protocol.reader.Reader method), 22
 EXTENDED_INFO_TAG (aliro_actuator.access_protocol.defines.Select attribute), 15
F
 FAST (aliro_actuator.access_protocol.apdu.Transaction attribute), 12
 FCI_TAG (aliro_actuator.access_protocol.defines.Select attribute), 15
 FILE_OR_APP_NOT_FOUND (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 11
 FINISHED_WITH_FAILURE (aliro_actuator.access_protocol.apdu.SI attribute), 11
 FINISHED_WITH_SUCCESS (aliro_actuator.access_protocol.apdu.SI attribute), 11
 from_bytes() (aliro_actuator.access_protocol.tlv.TLV static method), 24
 FUNCTION_NOT_SUPPORTED (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12
 FUNCTIONS_IN_CLA_NOT_SUPPORTED (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12
G
 generate_ephemeral_key() (aliro_actuator.access_protocol.reader.ReaderSession method), 24
 generate_ephemeral_key() (aliro_actuator.access_protocol.user_device.UserSession method), 24

method), 30
 generate_public_key()
 (aliro_actuator.trust_framework.key.PrivateKey
 method), 38
 GENERIC_ERROR(aliro_actuator.access_protocol.apdu.StatusBytes
 attribute), 12
 get_all_bytes_of_tag()
 (aliro_actuator.access_protocol.tlv.TLV
 method), 25
 get_all_of_tag()(aliro_actuator.access_protocol.tlv.TLV
 method), 25
 get_bytes()(aliro_actuator.access_protocol.tlv.TLV
 method), 25
 get_data()(aliro_actuator.access_protocol.mailbox.Mailbox
 method), 19
 get_endpoint_ephemeral_key()
 (aliro_actuator.access_protocol.reader.ReaderSession
 method), 24
 get_endpoint_epubkey()
 (aliro_actuator.access_protocol.user_device.UserDevice
 method), 30
 get_endpoint_public_key()
 (aliro_actuator.trust_framework.endpoint.Endpoint
 method), 36
 get_endpoint_reader_public_key()
 (aliro_actuator.access_protocol.user_device.UserDevice
 method), 30
 get_message()(aliro_actuator.transport_protocol.ble_uart.BLEUART
 method), 34
 get_message()(aliro_actuator.transport_protocol.nfc.NFC
 method), 35
 get_message()(aliro_actuator.transport_protocol.socket.Socket
 method), 35
 get_message()(aliro_actuator.transport_protocol.TransportProtocolBase
 method), 35
 get_private_key()(aliro_actuator.trust_framework.key.KeyPair
 method), 37
 get_public_key()(aliro_actuator.trust_framework.certificate.Certificate
 method), 36
 get_public_key()(aliro_actuator.trust_framework.key.KeyPair
 method), 37
 get_public_key_as_bytes()
 (aliro_actuator.trust_framework.key.KeyPair
 method), 37
 get_raw()(aliro_actuator.access_protocol.mailbox.Mailbox
 method), 20
 get_reader_epubkey()
 (aliro_actuator.access_protocol.reader.ReaderSession
 method), 24
 get_reader_public_key()
 (aliro_actuator.access_protocol.user_device.UserDevice
 method), 30
 get_reader_public_key()
 (aliro_actuator.trust_framework.endpoint.Endpoint
 method), 36
 GET_RESPONSE(aliro_actuator.access_protocol.apdu.INS
 attribute), 10
 get_tlv()(aliro_actuator.access_protocol.mailbox.Mailbox
 method), 20
 get_tlv()(aliro_actuator.access_protocol.tlv.TLV
 method), 25
 get_transaction_type()
 (aliro_actuator.access_protocol.user_device.UserSession
 method), 30
 get_value()(aliro_actuator.access_protocol.tlv.TLV
 method), 25
 get_x()(aliro_actuator.trust_framework.key.PublicKey
 method), 38
 get_y()(aliro_actuator.trust_framework.key.PublicKey
 method), 38
 Global(class in aliro_actuator), 39
 H
 handle(aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t
 attribute), 33
 handle_auth0()(aliro_actuator.access_protocol.reader.Reader
 method), 22
 handle_auth0()(aliro_actuator.access_protocol.user_device.UserDevice
 method), 27
 handle_auth1()(aliro_actuator.access_protocol.reader.Reader
 method), 22
 handle_auth1()(aliro_actuator.access_protocol.user_device.UserDevice
 method), 27
 handle_control_flow()
 (aliro_actuator.access_protocol.reader.Reader
 method), 22
 handle_control_flow()
 (aliro_actuator.access_protocol.user_device.UserDevice
 method), 27
 handle_exchange()(aliro_actuator.access_protocol.reader.Reader
 method), 23
 handle_exchange()(aliro_actuator.access_protocol.user_device.UserDevice
 method), 27
 handle_load_cert()(aliro_actuator.access_protocol.reader.Reader
 method), 23
 handle_load_cert()(aliro_actuator.access_protocol.user_device.UserDevice
 method), 27
 handle_select()(aliro_actuator.access_protocol.reader.Reader
 method), 23
 handle_select()(aliro_actuator.access_protocol.user_device.UserDevice
 method), 27
 has_identifier()(aliro_actuator.trust_framework.endpoint.Endpoint
 method), 36
 INCORRECT_P1_P2(aliro_actuator.access_protocol.apdu.StatusBytes
 attribute), 12

INCORRECT_PARAMETERS_IN_DATA (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12	KEY_SLOT_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 14
INCORRECT_SECURE_MESSAGING_DOS (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12	KeyLookupFailed, 19
INDEX_TAG (aliro_actuator.access_protocol.mailbox.Mailbox attribute), 19	KeyPair (class in aliro_actuator.trust_framework.key), 37
initialization() (aliro_actuator.transport_protocol.ble_uwb.BLEUWB method), 34	LOAD_CERT (aliro_actuator.access_protocol.apdu.INS at- tribute), 10
initialization() (aliro_actuator.transport_protocol.nfc.NFC method), 35	LOCK (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 12
initialization() (aliro_actuator.transport_protocol.socket.Socket method), 35	LOCK_USING_RKE (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 12
initialization() (aliro_actuator.transport_protocol.TransportProtocolBase method), 35	LOCK_WITH_FORCE_USER_AUTHENTICATION (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 12
initialize() (aliro_actuator.hw_driver.pn7160_driver.Driver method), 34	Logger (aliro_actuator.Global attribute), 39
INS (class in aliro_actuator.access_protocol.apdu), 10	LoggingConfigError, 39
InterceptHandler (class in aliro_actuator.log), 38	LOGICAL_CHANNEL_NOT_SUPPORTED (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12
INVALID_CLASS (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12	loglevel_mapping (aliro_actuator.log.InterceptHandler attribute), 39
INVALID_INSTRUCTION (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12	M
InvalidAIDError, 18	Mailbox (class in aliro_actuator.access_protocol.mailbox), 19
InvalidCLAError, 18	MAILBOX_DATA_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 14
InvalidCommandDataError, 18	MailboxFormatError, 20
InvalidCommandError, 18	MailboxPermissionError, 20
InvalidINSError, 18	MailboxSession (class in aliro_actuator.access_protocol.user_device), 26
InvalidKeyError, 37	MailboxSession.Command (class in aliro_actuator.access_protocol.user_device), 26
InvalidKeyFormatError, 37	main_loop() (aliro_actuator.access_protocol.user_device.UserDevice method), 27
InvalidLcError, 18	MASK_A (aliro_actuator.hw_driver.pn7160_driver.api.TECHNOLOGY_MAS tribute), 32
InvalidModeError, 34	MAX_COMMAND_TAG (aliro_actuator.access_protocol.defines.Select attribute), 15
InvalidParameterError, 18	max_ndef_length (aliro_actuator.hw_driver.pn7160_driver.api.ndef_info tribute), 32
InvalidResponseDataError, 18	MAX_RESPONSE_TAG (aliro_actuator.access_protocol.defines.Select attribute), 15
InvalidResponseError, 18	MEMORY_FAILURE (aliro_actuator.access_protocol.apdu.StatusBytes tribute), 12
InvalidStatusError, 18	Message (class in aliro_actuator.access_protocol.apdu), 10
is_def (aliro_actuator.hw_driver.pn7160_driver.api.ndef_info_t tribute), 32	MessageTooLongError, 19
is_started() (aliro_actuator.access_protocol.user_device.MailboxSession method), 26	Mode (class in aliro_actuator.transport_protocol), 35
is_writable (aliro_actuator.hw_driver.pn7160_driver.api.ndef_info_t tribute), 32	module
IssuerAuth (class in aliro_actuator.credential.access_credential), 31	
K	
Key (class in aliro_actuator.trust_framework.key), 37	
KEY_SLOT (aliro_actuator.access_protocol.apdu.Auth1Response tribute), 9	
KEY_SLOT_LEN (aliro_actuator.access_protocol.defines.Auth1 tribute), 14	

aliro_actuator, 39
 aliro_actuator.access_protocol, 30
 aliro_actuator.access_protocol.apdu, 7
 aliro_actuator.access_protocol.authentication, 13
 aliro_actuator.access_protocol.defines, 13
 aliro_actuator.access_protocol.encryption, 16
 aliro_actuator.access_protocol.errors, 18
 aliro_actuator.access_protocol.mailbox, 19
 aliro_actuator.access_protocol.reader, 21
 aliro_actuator.access_protocol.tlv, 24
 aliro_actuator.access_protocol.user_device, 26
 aliro_actuator.credential, 31
 aliro_actuator.credential.access_credential, 31
 aliro_actuator.credential.errors, 31
 aliro_actuator.credential.revocation_document, 31
 aliro_actuator.hw_driver, 34
 aliro_actuator.hw_driver.pn7160_driver, 34
 aliro_actuator.hw_driver.pn7160_driver.api, 32
 aliro_actuator.hw_driver.pn7160_driver.errors, 33
 aliro_actuator.log, 38
 aliro_actuator.transport_protocol, 35
 aliro_actuator.transport_protocol.ble_uwb, 34
 aliro_actuator.transport_protocol.errors, 34
 aliro_actuator.transport_protocol.nfc, 35
 aliro_actuator.transport_protocol.socket, 35
 aliro_actuator.trust_framework, 38
 aliro_actuator.trust_framework.certificate, 36
 aliro_actuator.trust_framework.endpoint, 36
 aliro_actuator.trust_framework.errors, 37
 aliro_actuator.trust_framework.key, 37

N

NCLError, 33
 NCINotFoundError, 33
 ndef_info_t (class in aliro_actuator.hw_driver.pn7160_driver.api), 32
 NFC (aliro_actuator.access_protocol.defines.TransportProtocol attribute), 15

NFC (class in aliro_actuator.transport_protocol.nfc), 35
 nfc_tag_info_t (class in aliro_actuator.hw_driver.pn7160_driver.api), 32
 nfcHostCardEmulationCallback_t (class in aliro_actuator.hw_driver.pn7160_driver.api), 32
 nfcTagCallback_t (class in aliro_actuator.hw_driver.pn7160_driver.api), 32
 NO_PRECISE_DIAGNOSIS (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12
 NoDataReceivedError, 33, 34
 NONE (aliro_actuator.access_protocol.apdu.S2 attribute), 11
 NoReaderError, 33
 NoTagError, 33
 NOTIFY_TAG (aliro_actuator.access_protocol.defines.Exchange attribute), 14

O

onDataReceived (aliro_actuator.hw_driver.pn7160_driver.api.nfcHostCardEmulation attribute), 32
 onHostCardEmulationActivated (aliro_actuator.hw_driver.pn7160_driver.api.nfcHostCardEmulation attribute), 32
 onHostCardEmulationDeactivated (aliro_actuator.hw_driver.pn7160_driver.api.nfcHostCardEmulation attribute), 32
 onTagArrival (aliro_actuator.hw_driver.pn7160_driver.api.nfcTagCallback attribute), 32
 onTagDeparture (aliro_actuator.hw_driver.pn7160_driver.api.nfcTagCallback attribute), 32

P

parse() (aliro_actuator.credential.access_credential.AccessCredential method), 31
 parse_as_auth0() (aliro_actuator.access_protocol.apdu.Command method), 9
 parse_as_auth0() (aliro_actuator.access_protocol.apdu.Response method), 10
 parse_as_auth1() (aliro_actuator.access_protocol.apdu.Command method), 9
 parse_as_auth1() (aliro_actuator.access_protocol.apdu.Response method), 10
 parse_as_control_flow() (aliro_actuator.access_protocol.apdu.Command method), 9
 parse_as_control_flow() (aliro_actuator.access_protocol.apdu.Response method), 10
 parse_as_envelope() (aliro_actuator.access_protocol.apdu.Command method), 9

`method`), 9
`parse_as_envelope()`
 (*aliro_actuator.access_protocol.apdu.Response*
 method), 11
`parse_as_exchange()`
 (*aliro_actuator.access_protocol.apdu.Command*
 method), 9
`parse_as_exchange()`
 (*aliro_actuator.access_protocol.apdu.Response*
 method), 11
`parse_as_get_response()`
 (*aliro_actuator.access_protocol.apdu.Command*
 method), 9
`parse_as_get_response()`
 (*aliro_actuator.access_protocol.apdu.Response*
 method), 11
`parse_as_load_cert()`
 (*aliro_actuator.access_protocol.apdu.Command*
 method), 9
`parse_as_load_cert()`
 (*aliro_actuator.access_protocol.apdu.Response*
 method), 11
`parse_as_select()` (*aliro_actuator.access_protocol.apdu.Command*
 method), 9
`parse_as_select()` (*aliro_actuator.access_protocol.apdu.Response*
 method), 11
`parse_command()` (*aliro_actuator.access_protocol.apdu.APDU*
 method), 8
`parse_response()` (*aliro_actuator.access_protocol.apdu.APDU*
 method), 8
 PN7160DriverError, 33
 PrivateKey (class in *aliro_actuator.trust_framework.key*), 37
 PROPRIETARY_TAG (*aliro_actuator.access_protocol.defines*.*Auth0*
 attribute), 15
 protocol (*aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info*
 attribute), 33
 PROTOCOL_VERSION_NOT_SUPPORTED
 (*aliro_actuator.access_protocol.apdu.S2*
 attribute), 11
 PublicKey (class in *aliro_actuator.trust_framework.key*), 38
R
 RE_VENDOR_SPECIFIC_TAG
 (*aliro_actuator.access_protocol.defines.Auth0*
 attribute), 13
`read()` (*aliro_actuator.access_protocol.mailbox.Mailbox*
 method), 20
 READ_TAG (*aliro_actuator.access_protocol.defines.Exchange*
 attribute), 14
 READER (*aliro_actuator.access_protocol.encryption.DeviceType*
 attribute), 16
 attribute), 35
 Reader (class in *aliro_actuator.access_protocol.reader*), 21
 READER_EPUBK_TAG (*aliro_actuator.access_protocol.defines.Auth0*
 attribute), 13
 READER_EPUBK_TAG (*aliro_actuator.access_protocol.defines.EndpointAuth*
 attribute), 14
 READER_EPUBK_TAG (*aliro_actuator.access_protocol.defines.ReaderAuth*
 attribute), 15
 READER_IDENTIFIER_TAG
 (*aliro_actuator.access_protocol.defines.Auth0*
 attribute), 13
 READER_IDENTIFIER_TAG
 (*aliro_actuator.access_protocol.defines.EndpointAuth*
 attribute), 14
 READER_IDENTIFIER_TAG
 (*aliro_actuator.access_protocol.defines.ReaderAuth*
 attribute), 15
 READER_SIG_TAG (*aliro_actuator.access_protocol.defines.Auth1*
 attribute), 14
 ReaderAuth (class in *aliro_actuator.access_protocol.defines*),
 in
 ReaderSession (class in
 aliro_actuator.access_protocol.reader), 24
 receive_message() (*aliro_actuator.hw_driver.pn7160_driver.Driver*
 method), 34
 Response (class in *aliro_actuator.access_protocol.apdu*),
 10
 response_auth0() (*aliro_actuator.access_protocol.user_device.UserDev*
 method), 28
 response_auth1() (*aliro_actuator.access_protocol.user_device.UserDev*
 method), 28
 response_control_flow()
 (*aliro_actuator.access_protocol.user_device.UserDevice*
 method), 28
 response_envelope()
 (*aliro_actuator.access_protocol.user_device.UserDevice*
 method), 28
 response_error() (*aliro_actuator.access_protocol.user_device.UserDev*
 method), 28
 response_exchange()
 (*aliro_actuator.access_protocol.user_device.UserDevice*
 method), 28
 response_get_response()
 (*aliro_actuator.access_protocol.user_device.UserDevice*
 method), 28
 response_load_cert()
 (*aliro_actuator.access_protocol.user_device.UserDevice*
 method), 28
 response_select() (*aliro_actuator.access_protocol.user_device.UserDev*
 method), 29
 retrieve() (*aliro_actuator.credential.Credential*
 method), 31

return_exchange_error_and_close_channel() (aliro_actuator.access_protocol.user_device.UserDevice method), 29
 REVOCATION_TIMESTAMP_LEN (aliro_actuator.access_protocol.defines.Auth1 attribute), 14
 REVOCATION_TIMESTAMP_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 14
 RevocationDocument (class in aliro_actuator.credential.revocation_document), 31
S
 S1 (class in aliro_actuator.access_protocol.apdu), 11
 S1_TAG (aliro_actuator.access_protocol.defines.ControlFlow attribute), 14
 S2 (class in aliro_actuator.access_protocol.apdu), 11
 S2_TAG (aliro_actuator.access_protocol.defines.ControlFlow attribute), 14
 SECURITY_STATUS_NOT_SATISFIED (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12
 SELECT (aliro_actuator.access_protocol.apdu.INS attribute), 10
 Select (class in aliro_actuator.access_protocol.defines), 15
 SELECT_DONE (aliro_actuator.access_protocol.user_device.UserSession attribute), 30
 send_message() (aliro_actuator.hw_driver.pn7160_driver.Driver method), 34
 send_message() (aliro_actuator.transport_protocol.ble_uwb.BLEUWB attribute), 34
 send_message() (aliro_actuator.transport_protocol.nfc.NFC attribute), 35
 send_message() (aliro_actuator.transport_protocol.socket.Socket attribute), 35
 send_message() (aliro_actuator.transport_protocol.TransportProtocol attribute), 35
 SESSION_START (aliro_actuator.access_protocol.user_device.UserSession attribute), 30
 SessionError, 19
 SET (aliro_actuator.access_protocol.user_device.MailboxSession.ControlFlow attribute), 26
 set() (aliro_actuator.access_protocol.mailbox.Mailbox attribute), 20
 set_auth0_data() (aliro_actuator.access_protocol.user_device.UserSession attribute), 30
 set_auth1_info() (aliro_actuator.access_protocol.reader.ReaderSession attribute), 24
 set_cert_and_verify() (aliro_actuator.access_protocol.user_device.UserSession method), 30
 set_endpoint() (aliro_actuator.access_protocol.user_device.UserSession method), 30
 set_endpoint_ephemeral_key() (aliro_actuator.access_protocol.reader.ReaderSession method), 24
 set_flag() (aliro_actuator.access_protocol.reader.ReaderSession method), 24
 set_logger() (aliro_actuator.Global class method), 39
 set_reader_public_key() (aliro_actuator.access_protocol.user_device.UserSession method), 30
 set_select_info() (aliro_actuator.access_protocol.reader.ReaderSession method), 24
 set_shared_key() (aliro_actuator.access_protocol.reader.ReaderSession method), 24
 set_shared_key() (aliro_actuator.access_protocol.user_device.UserSession method), 30
 SET_TAG (aliro_actuator.access_protocol.defines.Exchange attribute), 14
 sign() (aliro_actuator.trust_framework.endpoint.Endpoint method), 36
 sign() (aliro_actuator.trust_framework.key.KeyPair method), 37
 sign() (aliro_actuator.trust_framework.key.PrivateKey method), 38
 SIGNALING_BITMAP_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 14
 Socket (class in aliro_actuator.transport_protocol.socket), 35
 SOCKET_BLE (aliro_actuator.access_protocol.defines.TransportProtocol attribute), 15
 SOCKET_NFC (aliro_actuator.access_protocol.defines.TransportProtocol attribute), 15
 STANDARD (aliro_actuator.access_protocol.apdu.Transaction attribute), 12
 start() (aliro_actuator.access_protocol.user_device.MailboxSession attribute), 26
 start_new_session() (aliro_actuator.access_protocol.reader.ReaderSession method), 23
 start_new_session() (aliro_actuator.access_protocol.user_device.UserDevice method), 29
 START_OF_DATA_TAG (aliro_actuator.access_protocol.mailbox.Mailbox attribute), 19
 start_upaid() (aliro_actuator.access_protocol.user_device.UserSession method), 30
 StatusBytes (class in aliro_actuator.access_protocol.apdu), 11
 step_up_aid_select_required() (aliro_actuator.access_protocol.reader.ReaderSession method), 24
 step_up_transaction()

(aliro_actuator.access_protocol.reader.Reader method), 23
 stop() (aliro_actuator.access_protocol.user_device.Mailbox method), 26
 store() (aliro_actuator.credential.Credential method), 31
 SUCCESS (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12
 support_extended_length_apdu (aliro_actuator.access_protocol.Device property), 30
 sw1 (aliro_actuator.access_protocol.apdu.Response property), 11
 sw2 (aliro_actuator.access_protocol.apdu.Response property), 11
T
 technology (aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t attribute), 33
 TECHNOLOGY_MASK (class in aliro_actuator.hw_driver.pn7160_driver.api), 32
 TLV (class in aliro_actuator.access_protocol.tlv), 24
 TlvError, 26
 to_bytes() (aliro_actuator.access_protocol.apdu.Message method), 10
 to_bytes() (aliro_actuator.access_protocol.tlv.TLV method), 26
 to_data() (aliro_actuator.access_protocol.tlv.TLV method), 26
 Transaction (class in aliro_actuator.access_protocol.apdu), 12
 TRANSACTION_CODE_TAG (aliro_actuator.access_protocol.defines.Auth0 attribute), 13
 TRANSACTION_ID_TAG (aliro_actuator.access_protocol.defines.Auth0 attribute), 13
 transaction_identifier (aliro_actuator.access_protocol.reader.ReaderSession property), 24
 TRANSACTION_IDENTIFIER_TAG (aliro_actuator.access_protocol.defines.EndpointAuth attribute), 14
 TRANSACTION_IDENTIFIER_TAG (aliro_actuator.access_protocol.defines.ReaderAuth attribute), 15
 transaction_initiation() (aliro_actuator.access_protocol.reader.Reader method), 23
 transaction_initiation() (aliro_actuator.access_protocol.user_device.UserDevice method), 29
 TransactionCode (class in aliro_actuator.access_protocol.apdu), 12
 TransportProtocol (class in aliro_actuator.access_protocol.defines), 15
 TransportProtocolBase (class in aliro_actuator.transport_protocol), 35
 TransportProtocolError, 34
 TrustFrameworkError, 37
 TYPE_TAG (aliro_actuator.access_protocol.defines.Select attribute), 15
U
 uid (aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t attribute), 33
 uid_length (aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t attribute), 33
 UnexpectedCommandError, 19
 UnexpectedResponseError, 19
 UNLOCK (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 12
 UNLOCK_USING_RKE (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 12
 UNLOCK_WITH_FORCE_USER_AUTHENTICATION (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 12
 UPDATE_DOC_TAG (aliro_actuator.access_protocol.defines.Exchange attribute), 14
 update_state() (aliro_actuator.access_protocol.user_device.UserSession method), 30
 URSK_TAG (aliro_actuator.access_protocol.defines.Exchange attribute), 14
 USAGE (aliro_actuator.access_protocol.defines.EndpointAuth attribute), 14
 USAGE (aliro_actuator.access_protocol.defines.ReaderAuth attribute), 15
 USAGE_TAG (aliro_actuator.access_protocol.defines.EndpointAuth attribute), 14
 USAGE_TAG (aliro_actuator.access_protocol.defines.ReaderAuth attribute), 15
 USER (aliro_actuator.access_protocol.encryption.DeviceType attribute), 16
 UserDevice (class in aliro_actuator.access_protocol.user_device), 26
 UserSession (class in aliro_actuator.access_protocol.user_device), 29
 UserSessionState (class in aliro_actuator.access_protocol.user_device), 30
V
 validate_credential() (aliro_actuator.credential.access_credential.AccessCredential method), 31
 VENDOR_SPECIFIC_TAG (aliro_actuator.access_protocol.defines.Auth0

attribute), 13
 VENDOR_SPECIFIC_TAG
 (*aliro_actuator.access_protocol.defines.Select*
 attribute), 15
 VerificationError, 16
 verify() (*aliro_actuator.trust_framework.certificate.Certificate*
 method), 36
 verify() (*aliro_actuator.trust_framework.key.KeyPair*
 method), 37
 verify() (*aliro_actuator.trust_framework.key.PublicKey*
 method), 38
 VersionError, 19

W

wait_for_command() (*aliro_actuator.access_protocol.user_device.UserDevice*
 method), 29
 wait_for_connection()
 (*aliro_actuator.transport_protocol.ble_uwb.BLEUWB*
 method), 34
 wait_for_connection()
 (*aliro_actuator.transport_protocol.nfc.NFC*
 method), 35
 wait_for_connection()
 (*aliro_actuator.transport_protocol.socket.Socket*
 method), 35
 wait_for_connection()
 (*aliro_actuator.transport_protocol.TransportProtocolBase*
 method), 35
 wait_for_reader() (*aliro_actuator.hw_driver.pn7160_driver.Driver*
 method), 34
 wait_for_tag() (*aliro_actuator.hw_driver.pn7160_driver.Driver*
 method), 34
 WRITE (*aliro_actuator.access_protocol.user_device.MailboxSession.Command*
 attribute), 26
 write() (*aliro_actuator.access_protocol.mailbox.Mailbox*
 method), 20
 WRITE_TAG (*aliro_actuator.access_protocol.defines.Exchange*
 attribute), 14
 WRONG_LENGTH_IN_LC (*aliro_actuator.access_protocol.apdu.StatusBytes*
 attribute), 12